

Inserting Data

In this chapter, you will learn how to insert data into tables by using the SQL `INSERT` statement.

Understanding Data Insertion

`SELECT` is undoubtedly the most frequently used SQL statement (which is why the past 15 chapters are dedicated to it). But there are three other frequently used SQL statements that you should learn. The first one is `INSERT`. (You'll get to the other two in the next chapter.)

As its name suggests, `INSERT` is used to insert (add) rows to a database table. `INSERT` can be used in several ways:

- To insert a single complete row
- To insert a single partial row
- To insert multiple rows
- To insert the results of a query

We'll now look at each of these.

Note

INSERT and System Security Use of the `INSERT` statement can be disabled per table or per user by using MySQL security, as explained in Chapter 28, “Managing Security.”

Inserting Complete Rows

The simplest way to insert data into a table is to use the basic `INSERT` syntax, which requires that you specify the table name and the values to be inserted into the new row. Here is an example of this:

► Input

```
INSERT INTO Customers  
VALUES(NULL,  
      'Pep E. LaPew',
```

```
'100 Main Street',  
'Los Angeles',  
'CA',  
'90046',  
'USA',  
NULL,  
NULL);
```

Note

No Output INSERT statements usually generate no output.

► Analysis

This example inserts a new customer into the `customers` table. The data to be stored in each table column is specified in the `VALUES` clause, and a value must be provided for every column. If a column has no value (as is the case for the `cust_contact` and `cust_email` columns), the `NULL` value should be used (assuming that the table allows no value to be specified for that column). The columns must be populated in the order in which they appear in the table definition. The first column, `cust_id`, is also `NULL`. This is because MySQL automatically increments that column each time a row is inserted. You wouldn't want to specify a value (that is MySQL's job), and you can't omit the column (because, as mentioned earlier, every column must be listed). Therefore, a `NULL` value is specified; MySQL ignores it and inserts the next available `cust_id` value in its place.

Although this syntax is indeed simple, it is not at all safe and should generally be avoided at all costs. This SQL statement is highly dependent on the order in which the columns are defined in the table. It also depends on information about that order being readily available. Even if the information is available, there is no guarantee that the columns will be in exactly the same order the next time the table is reconstructed. Therefore, writing SQL statements that depend on specific column ordering is very unsafe. If you try to do it, something will inevitably break at some point.

The safer (and unfortunately more cumbersome) way to write the `INSERT` statement is as follows:

► Input

```
INSERT INTO customers(cust_name,  
    cust_address,  
    cust_city,  
    cust_state,  
    cust_zip,  
    cust_country,  
    cust_contact,  
    cust_email)  
VALUES('Pep E. LaPew',  
    '100 Main Street',  
    'Los Angeles',  
    'CA',
```

```
'90046',  
'USA',  
NULL,  
NULL);
```

► Analysis

This example does exactly the same thing as the previous `INSERT` statement, but this time, the column names are explicitly stated in parentheses after the table name. When the row is inserted, MySQL matches each item in the columns list with the appropriate value in the `VALUES` list. The first entry in `VALUES` corresponds to the first specified column name. The second value corresponds to the second column name, and so on.

Because column names are provided, the items in the `VALUES` list must match the specified column names in the order in which they are specified—and not necessarily in the order that the columns appear in the table. The advantage of this is that, even if the table layout changes, the `INSERT` statement will still work correctly.

Notice that the `NULL` for `cust_id` is not needed, the `cust_id` column is not listed in the column list, and so no value is needed.

The following `INSERT` statement populates all the row columns (just as before), but it does so in a different order. Because the column names are specified, the insertion will work correctly:

► Input

```
INSERT INTO customers(cust_name,  
    cust_contact,  
    cust_email,  
    cust_address,  
    cust_city,  
    cust_state,  
    cust_zip,  
    cust_country)  
VALUES('Pep E. LaPew',  
    NULL,  
    NULL,  
    '100 Main Street',  
    'Los Angeles',  
    'CA',  
    '90046',  
    'USA');
```

Tip

Always Use a Columns List Avoid using `INSERT` without explicitly specifying the columns list. Using a columns list will greatly increase the probability that your SQL will continue to function in the event that table changes occur.

Caution

Using VALUES Carefully Regardless of the `INSERT` syntax being used, the correct number of items must be specified in the `VALUES` list. If no column names are provided, a value must be present for every table column. If columns names are provided, a value must be present for each listed column. If none is present, an error message will be generated, and the row will not be inserted.

Using this syntax, you can also omit columns. This means you only provide values for some columns and not for others. (You've actually already seen an example of this; `cust_id` was omitted when column names were explicitly listed.)

Caution

Omitting Columns You may omit columns from an `INSERT` operation if the table definition allows it. One of the following conditions must exist:

- The column is defined as allowing `NULL` values (no value at all).
- A default value is specified in the table definition. This means the default value will be used if no value is specified.

If you omit a value from a table that does not allow `NULL` values and does not have a default, MySQL generates an error message, and the row is not inserted.

Tip

Improve Overall Performance Databases are frequently accessed by multiple clients, and it is MySQL's job to manage which requests are processed and in which order. `INSERT` operations can be time-consuming (especially if there are many indexes to be updated), and this can hurt the performance of `SELECT` statements that are waiting to be processed.

If data retrieval is of utmost importance (as it usually is), you can instruct MySQL to lower the priority of your `INSERT` statement by adding the keyword `LOW_PRIORITY` in between `INSERT` and `INTO`, like this:

```
INSERT LOW_PRIORITY INTO
```

Incidentally, this also applies to the `UPDATE` and `DELETE` statements that you'll learn about in the next chapter.

Inserting Multiple Rows

INSERT inserts a single row into a table. But what if you need to insert multiple rows? You can simply use multiple INSERT statements, and you can even submit them all at once, with each one terminated by a semicolon, like this:

► Input

```
INSERT INTO customers(cust_name,  
    cust_address,  
    cust_city,  
    cust_state,  
    cust_zip,  
    cust_country)  
VALUES('Pep E. LaPew',  
    '100 Main Street',  
    'Los Angeles',  
    'CA',  
    '90046',  
    'USA');  
INSERT INTO customers(cust_name,  
    cust_address,  
    cust_city,  
    cust_state,  
    cust_zip,  
    cust_country)  
VALUES('M. Martian',  
    '42 Galaxy Way',  
    'New York',  
    'NY',  
    '11213',  
    'USA');
```

Or, if the column names and order are identical in the INSERT statements, you can combine the statements as follows:

► Input

```
INSERT INTO customers(cust_name,  
    cust_address,  
    cust_city,  
    cust_state,  
    cust_zip,  
    cust_country)  
VALUES(  
    'Pep E. LaPew',  
    '100 Main Street',  
    'Los Angeles',  
    'CA',  
    '90046',  
    'USA'
```

```
    ),  
    (  
        'M. Martian',  
        '42 Galaxy Way',  
        'New York',  
        'NY',  
        '11213',  
        'USA'  
    );
```

► Analysis

Here a single `INSERT` statement has multiple sets of values, each enclosed within parentheses and separated by commas.

Tip

Improve INSERT Performance The technique just described can improve the performance of your database processing, as MySQL processes multiple insertions in a single `INSERT` faster than it processes multiple `INSERT` statements.

Inserting Retrieved Data

`INSERT` is usually used to add a row to a table by using specified values. There is another form of `INSERT` that can be used to insert the result of a `SELECT` statement into a table. This is known as `INSERT SELECT`, and, as its name suggests, it is made up of an `INSERT` statement and a `SELECT` statement.

Suppose you want to merge a list of customers from another table into your `customers` table. Instead of reading one row at a time and inserting it with `INSERT`, you can do the following:

Note

Instructions Needed for the Next Example The following example imports data from a table named `custnew` into the `customers` table. To try this example, you need to create and populate the `custnew` table first. The format of the `custnew` table should be the same as the format of the `customers` table described in Appendix B, “The Example Tables.” When populating `custnew`, be sure not to use `cust_id` values that were already used in `customers` (as the subsequent `INSERT` operation will fail if primary key values are duplicated); alternatively, you can just omit that column and have MySQL generate new values during the import process.

 Input

```
INSERT INTO customers(cust_id,  
    cust_contact,  
    cust_email,  
    cust_name,  
    cust_address,  
    cust_city,  
    cust_state,  
    cust_zip,  
    cust_country)  
SELECT cust_id,  
    cust_contact,  
    cust_email,  
    cust_name,  
    cust_address,  
    cust_city,  
    cust_state,  
    cust_zip,  
    cust_country  
FROM custnew;
```

 Analysis

This example uses `INSERT SELECT` to import all the data from `custnew` into `customers`. Instead of listing the values to be inserted, the `SELECT` statement retrieves them from `custnew`. Each column in the `SELECT` corresponds to a column in the specified columns list. How many rows will this statement insert? It depends on how many rows are in the `custnew` table. If the table is empty, no rows will be inserted (and no error will be generated because the operation is still valid). If the table does, in fact, contain data, all that data is inserted into `customers`.

This example imports `cust_id` (and assumes that you have ensured that `cust_id` values are not duplicated). You could also simply omit that column (from both the `INSERT` and the `SELECT`), and MySQL generates new values.

Tip

Column Names in INSERT SELECT This example uses the same column names in both the `INSERT` and `SELECT` statements for simplicity's sake. But there is no requirement that says the column names must match. In fact, MySQL does not even pay attention to the column names returned by `SELECT`. Rather, the column position is used, so the first column in the `SELECT` statement (regardless of its name) is used to populate the first specified table column, and so on. This is very useful when you're importing data from tables that use different column names.

The `SELECT` statement used in an `INSERT SELECT` can include a `WHERE` clause to filter the data to be inserted.

Note

More Examples Looking for more examples of the use of `INSERT`? See the example table population scripts (described in Appendix B) used to create the example tables for this book.

Summary

In this chapter, you learned how to use `INSERT` to insert rows into a database table. You also learned several other ways to use `INSERT` and why explicit column specification is preferred. You also learned how to use `INSERT SELECT` to import rows from another table. In the next chapter, you'll learn how to use `UPDATE` and `DELETE` to further manipulate table data.

Challenges

1. Using `INSERT` and specifying columns, add yourself to the `Customers` table.
Explicitly list the columns you are adding and use only the ones you need.
2. Using `INSERT SELECT`, make backup copies of your `Orders` and `OrderItems` tables.